hexagone
École Supérieure d'Informatique

# Gérer un projet

**1GEST** – Gestion de projet et outils collaboratifs

Chris CHEVALIER

hexagone
École Supérieure d'Informatique

# Sommaire


#1 Qu'est-ce qu'un projet ?

#2 Cadrer, planifier, suivre

#3 Les méthodes de gestion de projet

#4 Scrum et Kanban en pratique

# #1 Qu'est-ce qu'un projet ?

Gérer un projet

# Un exemple pour tout le cours

Le bureau des étudiants organise un **tournoi Mario Kart** pour la rentrée. Il me demande une application web pour :

- inscrire les joueurs ;
- afficher la grille des courses ;
- saisir les scores et le classement en direct.

### Ce que je sais au départ

- Le tournoi a lieu dans **6 semaines**
- Nous sommes **trois** : Chris, Manon et Cosmo
- Le bureau des étudiants n'a **pas de budget**
- Le reste… il va falloir le demander

> Est-ce un projet ? Qu'est-ce qui le distingue de votre travail habituel ?

# Définition

Un **projet** est un ensemble d'actions menées pour répondre à un besoin, avec :

**Un objectif**

Un résultat précis à atteindre, attendu par quelqu'un

**Un début et une fin**

Une date de livraison connue dès le départ

**Des ressources limitées**

Du temps, des personnes, un budget

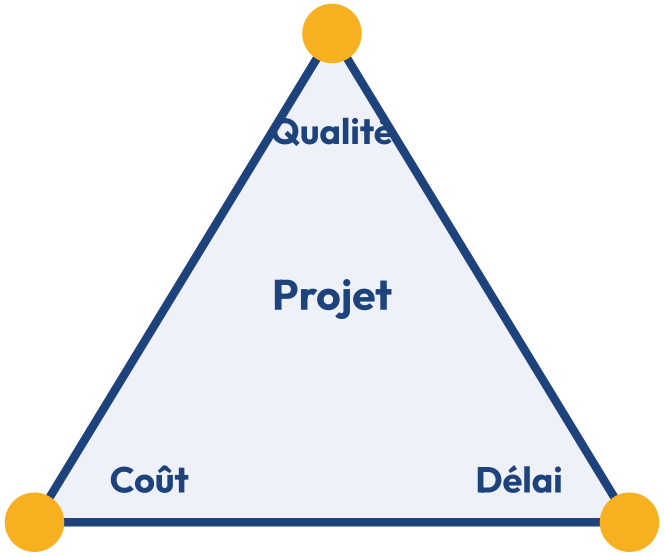
> Un projet est **unique**. Répondre aux e-mails du support tous les jours est une **activité récurrente**, pas un projet.

# Objectifs, livrables et jalons

| Notion | Définition | Pour le tournoi |
|---|---|---|
| **Objectif** | Ce que le projet doit permettre, mesurable et daté | Inscrire 64 joueurs avant le 1er octobre |
| **Livrable** | Un résultat concret remis au client : logiciel, document, maquette… | L'application en ligne, son mode d'emploi |
| **Jalon** | Une date clé où l'on vérifie qu'une étape est terminée | Maquettes validées le 15 septembre |

Un bon objectif est **SMART** : spécifique, mesurable, atteignable, réaliste et temporellement défini. « Faire une super appli » n'est pas un objectif.

# Le triangle d'or




- **Qualité** : le périmètre et le niveau d'exigence
- **Coût** : le budget et les personnes mobilisées
- **Délai** : la date de livraison

> Si l'un bouge, les autres bougent. Le client veut plus de fonctionnalités sans décaler la date ? Il faut plus de monde… ou renoncer à autre chose.

# Les acteurs d'un projet

### Le client · MOA

La **maîtrise d'ouvrage** exprime le besoin, valide les livrables et paie. Ici, le bureau des étudiants.

### L'équipe · MOE

La **maîtrise d'œuvre** réalise le projet : chef de projet, développeurs, designers, testeurs. Ici, Chris, Manon et Cosmo.

### Les utilisateurs

Ceux qui utiliseront le résultat. Ce ne sont pas toujours les clients ! Ici, les joueurs et les arbitres.

> Le **chef de projet** fait le lien entre tous : il planifie, coordonne l'équipe, anime les réunions et rend compte au client.

MOA et MOE sont des sigles du bâtiment : le maître d'ouvrage commande la maison, le maître d'œuvre la construit.

# Chacun voit le projet à sa façon

| Qui | Ce qu'il a en tête quand il dit « une appli pour le tournoi » |
|---|---|
| **Le bureau des étudiants** | Un beau site aux couleurs de l'école, avec les photos de l'an dernier |
| **Les arbitres** | Saisir un score en deux clics sur leur téléphone, entre deux courses |
| **Les joueurs** | Savoir quand ils jouent et contre qui |
| **Cosmo, développeur** | Tester enfin le dernier framework à la mode |

> Le premier travail d'une équipe est d'**écouter** puis de **reformuler** le besoin, jusqu'à ce que tout le monde parle du même projet.

# #2 Cadrer, planifier, suivre

Gérer un projet

# Le cycle de vie d'un projet

**Cadrage**
Comprendre le besoin, fixer le périmètre, estimer, planifier

→

**Conception**
Maquettes, choix techniques

→

**Réalisation**
Développement des fonctionnalités

→

**Recette**
Le client vérifie que tout fonctionne comme prévu

→

**Livraison**
Mise en ligne et passage de relais

→

**Bilan**
Ce qu'on garde et ce qu'on change

> On ne démarre pas la réalisation tant que le besoin, les objectifs et le périmètre ne sont pas clairs. **Une heure de cadrage économise des jours de travail.**

Recette : phase où le client teste le livrable et l'accepte (ou non) avant sa mise en service.

# Comprendre le besoin : le QQOQCCP

| Question | Ce qu'on cherche | Pour le tournoi |
|---|---|---|
| **Quoi ?** | Le produit ou le service attendu | Une application web d'inscription et de scores |
| **Qui ?** | Le client, les utilisateurs, l'équipe | Bureau des étudiants, joueurs, arbitres |
| **Où ?** | Le lieu, le support | Sur téléphone, dans la salle du tournoi |
| **Quand ?** | Les échéances, les jalons | En ligne une semaine avant le tournoi |
| **Comment ?** | Les moyens, les contraintes | Gratuit, hébergé par l'école |
| **Combien ?** | Le budget, les quantités | 0 €, 64 joueurs, 16 courses |
| **Pourquoi ?** | La raison d'être du projet | Fini les tableaux papier illisibles de l'an dernier |

# Le périmètre

Le **périmètre** (ou *scope*) dit ce que le projet fera… et ce qu'il **ne fera pas**.

### Dans le périmètre

- Inscription des joueurs
- Grille des courses
- Saisie des scores par les arbitres
- Classement en direct

### Hors périmètre

- Paiement des inscriptions
- Application mobile à installer
- Diffusion vidéo des courses
- Galerie de photos

> Écrire le hors périmètre évite les « Ah, mais je pensais que… » trois jours avant la livraison.

# Découper le travail

Un gros objectif est impossible à estimer. On le découpe jusqu'à obtenir des **tâches** qu'une personne peut réaliser en quelques heures ou quelques jours.

**Projet**
Application du tournoi

→

**Fonctionnalité**
Saisie des scores

→

**Tâches**
Maquette de l'écran · formulaire de score · calcul du classement · test avec un arbitre

> Une tâche bien découpée a un **verbe d'action**, un **responsable** et un **critère de fin** : « Manon dessine la maquette de l'écran des scores, validée par un arbitre ».

# Estimer

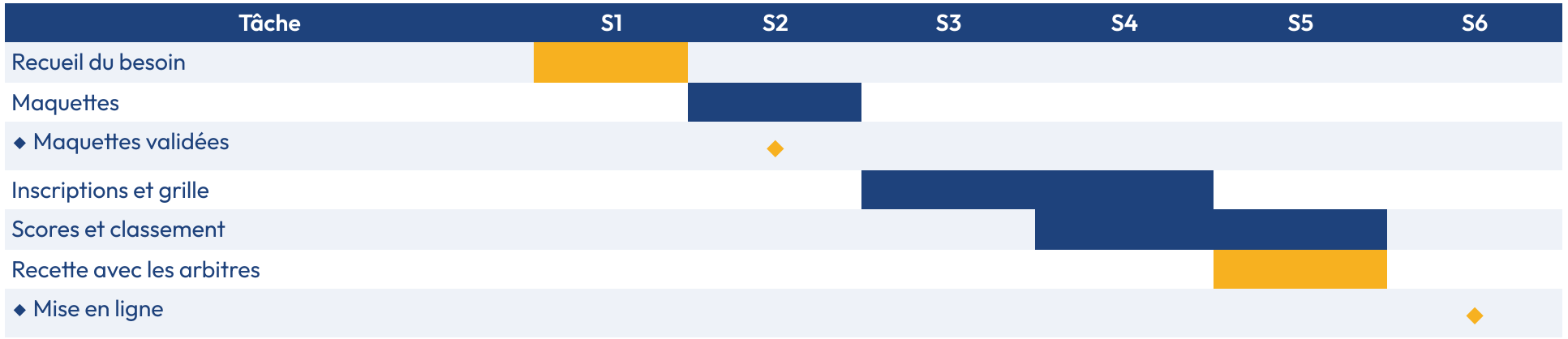
- Estimer, c'est prévoir le **temps** ou l'**effort** de chaque tâche
- On estime bien ce qu'on a déjà fait. Le reste est une supposition
- On estime **à plusieurs** : chacun voit des difficultés différentes
- On garde une **marge** pour l'imprévu (malade, bug, changement d'avis du client)

### Deux unités courantes

- **Jours-homme** : une personne pendant une journée
- **Points** : un effort relatif. Une tâche à 2 points demande à peu près deux fois plus d'effort qu'une tâche à 1 point

> Une estimation n'est pas une promesse. On la **révise** dès qu'on en sait plus.

# Planifier : le diagramme de Gantt




Le **Gantt** montre les tâches dans le temps, leurs **dépendances** (les scores attendent les maquettes) et les **jalons** ◆. Sur GitHub, la vue *Roadmap* en donne un équivalent.

# Suivre l'avancement et rendre compte

Chaque semaine, le chef de projet fait un **point d'étape** (ou *reporting*) au client, en quelques minutes :

| ☀️ **Dans les temps** | ⛅ **Sous surveillance** | 🌧️ **En difficulté** |
|---|---|---|
| Tout avance comme prévu | Un retard ou un risque, on a un plan | Une décision du client est nécessaire |

- **Ce qui est fait** depuis le dernier point
- **Ce qui reste à faire** d'ici le prochain jalon
- **Les blocages** et ce qu'on attend du client

# #3 Les méthodes de gestion de projet

Gérer un projet

# Le modèle en cascade

Les étapes s'enchaînent **dans un ordre fixe**. On ne passe à la suivante que lorsque la précédente est terminée et validée.




> Simple et rassurant quand le besoin est connu d'avance et ne change pas. On l'appelle aussi méthode **prédictive** : tout est prévu au départ.

# Le cycle en V

Une évolution de la cascade : chaque étape de conception correspond à une étape de **vérification**, préparée dès le départ.




> On descend la branche gauche (concevoir), on remonte la branche droite (vérifier). Très utilisé dans l'industrie, la banque ou la défense, où les exigences sont stables et les erreurs coûteuses.

Spécifications : description précise de ce que le logiciel doit faire. Tests unitaires : vérification de chaque petit morceau du programme pris isolément.

# Les limites des méthodes prédictives

- Le client découvre le produit **à la fin**, parfois après des mois : c'est l'**effet tunnel**
- Entre-temps, le besoin a changé : nouveau règlement, concurrent, idée en cours de route
- Changer d'avis coûte cher, car tout a été conçu d'un bloc

> Pour le tournoi : en semaine 4, le bureau des étudiants annonce qu'il y aura aussi une **catégorie en équipe**. En cascade, la conception est déjà figée. Que fait-on ?

# Le Manifeste agile (2001)

Dix-sept développeurs se réunissent pour proposer une autre façon de travailler. Ils écrivent quatre valeurs :

| Nous valorisons davantage… | … que |
|---|---|
| **Les individus et leurs interactions** | les processus et les outils |
| **Des logiciels opérationnels** | une documentation exhaustive |
| **La collaboration avec les clients** | la négociation contractuelle |
| **L'adaptation au changement** | le suivi d'un plan |

> Les éléments de droite ont de la valeur. Ceux de gauche en ont **davantage**. L'agilité n'est pas l'absence de plan ou de documentation.

# Itératif et incrémental

Une méthode agile livre le produit **par petits morceaux utilisables** à intervalles courts. Le plan est ajusté à chaque livraison.

**Itération 1**
Inscription des joueurs. Le bureau des étudiants l'essaie.

→

**Itération 2**
Grille des courses. Les joueurs donnent leur avis.

→

**Itération 3**
Scores, classement… et la catégorie en équipe, demandée entre-temps.

> À chaque itération, le client voit un produit qui fonctionne. S'il faut s'arrêter plus tôt, il a déjà **l'essentiel**.

# Prédictif ou agile ?

| Aspect | Cycle en V | Agile |
|---|---|---|
| **Besoin** | Connu et figé au départ | Précisé au fil du projet |
| **Livraison** | Une seule, à la fin | Régulière, toutes les 1 à 4 semaines |
| **Client** | Présent au début et à la fin | Impliqué tout au long du projet |
| **Changement** | Coûteux, à éviter | Attendu, intégré au plan |
| **Idéal pour** | Exigences stables, forte réglementation | Besoin incertain, produit innovant |

> Il n'y a pas de bonne méthode dans l'absolu. Beaucoup d'entreprises **combinent** les deux : un cadrage solide au départ puis des itérations courtes.

# #4 Scrum et Kanban en pratique

Gérer un projet

# Scrum en un coup d'œil

Scrum est la méthode agile la plus répandue. Le travail avance par **sprints** : des cycles de 1 à 4 semaines.




> Puis un nouveau sprint commence. Trois piliers : **transparence** (tout le monde voit l'état réel), **inspection** (on vérifie souvent), **adaptation** (on corrige vite).

*Scrum* vient du rugby : c'est la mêlée, où l'équipe avance ensemble. Le guide officiel tient en une quinzaine de pages.

# Les rôles dans Scrum

### Product Owner

Représente le client. Il écrit et **priorise** le backlog. Il décide de ce qui a le plus de valeur.

### Scrum Master

Garant de la méthode. Il anime les réunions et **lève les obstacles** qui bloquent l'équipe. Ce n'est pas un chef.

### Developers

Toute l'équipe qui réalise le produit : développeurs, mais aussi designers ou testeurs. Elle décide **comment** faire.

> Pour le tournoi : Manon est Product Owner, Cosmo Scrum Master. Tous les trois, moi compris, sommes Developers.

# La user story

Chaque élément du backlog est écrit du point de vue de l'utilisateur, en une phrase :

> **En tant qu'**arbitre,
> **je veux** saisir le résultat d'une course depuis mon téléphone
> **afin de** mettre à jour le classement sans attendre la fin du tournoi.

**Critères d'acceptation** : comment le client saura que c'est terminé.

- Je choisis la course puis la position de chaque joueur ;
- le classement est mis à jour en moins de 10 secondes ;
- je peux corriger une erreur de saisie.

# Prioriser le backlog : MoSCoW

| Priorité | Signification | Pour le tournoi |
|---|---|---|
| **Must** doit | Indispensable. Sans elle, le projet échoue | Inscription, grille, saisie des scores |
| **Should** devrait | Importante, mais on peut s'en passer au début | Classement en direct sur grand écran |
| **Could** pourrait | Un plus, si le temps le permet | Statistiques par personnage |
| **Won't** pas cette fois | Écartée explicitement pour cette version | Paiement en ligne |

> Le premier sprint ne contient que des **Must**. C'est le Product Owner qui tranche, avec le client.

# Estimer à plusieurs : le Planning Poker

| 1 | 2 | 3 | 5 | 8 | 13 | ? |
|---|---|---|---|---|---|---|

1. Le Product Owner lit une user story. L'équipe pose ses questions
2. Chacun choisit une carte **en secret**, puis tout le monde la retourne en même temps
3. Les estimations extrêmes s'expliquent : « j'ai mis 13 parce que… »
4. On revote jusqu'à se mettre d'accord

> Les cartes suivent la suite de Fibonacci : plus une tâche est grosse, plus l'estimation est incertaine. Une story à **13 ou plus** est trop grosse : on la découpe.

# Du backlog au sprint

- La **vélocité** est le nombre de points que l'équipe réalise en un sprint
- Au premier sprint, on ne la connaît pas : on fixe une **capacité** prudente
- Au **sprint planning**, on prend les stories les plus prioritaires jusqu'à remplir la capacité

### Pour le tournoi

- Capacité : 20 points par sprint
- Sprint 1 : inscription (5), grille (8), connexion des arbitres (5) = 18 points
- Il reste 2 points : on n'ajoute pas une story à 5

> Chaque fin de sprint est un **jalon** : le client voit et valide l'incrément.

# Les artefacts et la définition de « terminé »

**Product backlog**

La liste ordonnée de tout ce que le produit pourrait contenir. Elle évolue tout le temps

**Sprint backlog**

Les stories choisies pour le sprint et les tâches pour les réaliser

**Incrément**

Le produit utilisable obtenu à la fin du sprint

La **Definition of Done** est la liste de contrôle commune à toutes les stories : relue par un autre membre, testée, documentée, validée par le Product Owner. Une story à moitié faite n'est **pas terminée**.

# Les événements de Scrum

| Événement | Ce qu'on y fait | Durée max. (sprint de 2 semaines) |
|---|---|---|
| **Sprint planning** | Choisir les stories du sprint et les découper en tâches | 4 h |
| **Daily** | Chaque matin, debout : qu'ai-je fait hier, que fais-je aujourd'hui, qu'est-ce qui me bloque ? | 15 min |
| **Sprint review** | Montrer l'incrément au client et recueillir ses retours | 2 h |
| **Rétrospective** | L'équipe améliore sa façon de travailler | 1 h 30 |

> Chaque réunion a une durée maximale (*timebox*). Une réunion sans objectif ni limite de temps est une réunion perdue.

# Exemple : le daily du jour 3

| Qui | Hier | Aujourd'hui | Blocage |
|---|---|---|---|
| **Manon** | Maquette de l'écran d'inscription | Maquette de la grille | Aucun |
| **Cosmo** | Formulaire d'inscription | Enregistrement des joueurs | J'attends les règles du tournoi |
| **Chris** | Hébergement de l'application | J'aide Cosmo | Aucun |

> Le daily n'est **pas** une réunion pour résoudre les problèmes. Le blocage de Cosmo est noté. Cosmo, en tant que Scrum Master, appelle le bureau des étudiants juste après.

# Kanban : visualiser le flux




> Une carte par tâche, qui avance de gauche à droite. On **limite** le nombre de cartes « En cours » : on finit avant de commencer autre chose.

# Un tableau Kanban avec Trello

- Un **tableau** par projet, partagé avec l'équipe
- Une **liste** par colonne : Backlog, À faire, En cours, En revue, Terminé
- Une **carte** par user story, avec :
  - des membres (qui s'en occupe) ;
  - une étiquette (priorité, points) ;
  - une date limite ;
  - une checklist pour les critères d'acceptation.

### Les bonnes habitudes

- Le tableau reflète **la réalité**, pas les intentions
- Chacun déplace **ses** cartes, au fil de l'eau
- Une carte « En cours » a toujours un responsable
- On regarde le tableau ensemble à chaque daily

Trello est gratuit pour une petite équipe. Des alternatives libres existent, comme Kanboard ou Wekan, que l'on peut héberger soi-même.

# La rétrospective

À la fin de chaque sprint, l'équipe prend du recul sur **sa façon de travailler**, pas sur le produit.

### ▶ Commencer

Un daily à heure fixe, même à distance

### ■ Arrêter

Commencer une tâche sans la déplacer sur le tableau

### ↻ Continuer

Faire relire chaque maquette par un arbitre

> On repart avec **une ou deux actions concrètes**, chacune avec un responsable. Une rétrospective sans action ne sert à rien.

# Quelques questions

1. Quelle est la différence entre un livrable et un jalon ?
2. Qui est la MOA dans le projet du tournoi ? Et la MOE ?
3. Pourquoi le cycle en V convient-il mal à un besoin qui change ?
4. Quelle est la différence entre le product backlog et le sprint backlog ?
5. Pourquoi vote-t-on en secret au Planning Poker ?
6. Quel est le rôle du Scrum Master ? Est-ce le chef de l'équipe ?

# Cet après-midi : à vous de jouer

**1. Rôles**
Client, Product Owner, Scrum Master

→

**2. Besoin**
Le client explique, l'équipe reformule

→

**3. Backlog**
User stories et priorités

→

**4. Estimation**
Planning Poker, deux sprints

→

**5. Trello**
Le tableau de l'équipe

> À 15h30, chaque groupe fait un **point d'étape de 3 minutes** : où en êtes-vous, que reste-t-il, qu'est-ce qui bloque ?




# Des questions ?

Chris CHEVALIER

chris.chevalier@ecole-hexagone.com